



**Universidad Nacional Autónoma de
México**

Facultad de Ingeniería



Sistemas Distribuidos

Semestre 2027 - 1

**Laboratorio S7. Mexi Banco: de un monolito a
servicios que se encuentran solos**

Alumno: Fernández Herrera Mauricio

Equipo: Pumaverso Distribuido.

Profesor: ING. OSCAR MANUEL RUIZ HURTADO

Grupo: 02

Entrega: 09/10/2026

1.- Descarga y construcción del proyecto. 03/10/2026 – 10:52am

Para comenzar descargué el proyecto de Mexi-banco y realicé el primer levantamiento con Docker Compose. Con esto pude comprobar que el proyecto se construía correctamente y que los servicios podían iniciar antes de realizar los cambios correspondientes al laboratorio.

```
mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ docker images
```

Info →	In Use				
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA	
alpine:3.20	d9e853e87e55	12.2MB	3.71MB		
alpine:latest	28bd5fe8b56d	13MB	3.93MB		
hello-world:latest	5dd0d3e6e255	25.9kB	9.49kB		U
mexi-banco-app:latest	90fec4e603c5	381MB	119MB		U
mexi-banco-v06-cuenta:latest	85ee8e04b638	381MB	119MB		U
mexi-banco-v06-discovery:latest	9c1779169d58	393MB	125MB		
mexi-banco-v06-gateway:latest	a52c8d046815	384MB	120MB		

Figura 1.1. Primer levantamiento de Mexi-banco.

2.- El directorio: que cada servicio se anote solo. 06/10/2026 – 8:24pm

En esta parte configuré Eureka como directorio de servicios y agregué el cliente de Eureka a los servicios que debían registrarse. Después de reconstruir y levantar nuevamente el proyecto, los servicios comenzaron a registrarse automáticamente en el directorio.

Primero comprobé el estado de los contenedores y posteriormente revisé el tablero de Eureka, donde los cinco servicios aparecieron como UP. También detuve el servicio de notificación para comprobar que el directorio reflejara el cambio en el estado de los servicios.

La idea principal de esta parte es que los servicios ya no necesitan conocer directamente la dirección de otros servicios. En lugar de eso, pueden registrarse en el directorio y ser encontrados por su nombre.

```
mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STA
TUS					
mexi-banco-v06-cuenta-1	mexi-banco-v06-cuenta	"java -jar app.jar"	cuenta	52 seconds ago	Up
45 seconds (healthy)	0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp				
mexi-banco-v06-db-1	postgres:17-alpine	"docker-entrypoint.s..."	db	52 seconds ago	Up
51 seconds (healthy)	5432/tcp				
mexi-banco-v06-movimiento-1	mexi-banco-v06-movimiento	"java -jar app.jar"	movimiento	52 seconds ago	Up
45 seconds (healthy)	0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp				
mexi-banco-v06-notificacion-1	mexi-banco-v06-notificacion	"java -jar app.jar"	notificacion	52 seconds ago	Up
45 seconds (healthy)	0.0.0.0:8084->8080/tcp, [::]:8084->8080/tcp				
mexi-banco-v06-spei-1	mexi-banco-v06-spei	"java -jar app.jar"	spei	52 seconds ago	Up
45 seconds (healthy)	0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp				
mexi-banco-v06-transferencia-1	mexi-banco-v06-transferencia	"java -jar app.jar"	transferencia	52 seconds ago	Up
45 seconds (healthy)	0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp				

Figura 2.1. Volvemos a levantar el proyecto y verificamos contenedores.

```

[+] up 15/15
✔ Image mexi-banco-v06-movimiento Built 398.8s
✔ Image mexi-banco-v06-discovery Built 398.8s
✔ Image mexi-banco-v06-cuenta Built 398.8s
✔ Image mexi-banco-v06-spei Built 398.8s
✔ Image mexi-banco-v06-gateway Built 398.8s
✔ Image mexi-banco-v06-notificacion Built 398.8s
✔ Image mexi-banco-v06-transferencia Built 398.8s
✔ Container mexi-banco-v06-db-1 Healthy 13.1s
✔ Container mexi-banco-v06-discovery-1 Healthy 23.6s
✔ Container mexi-banco-v06-gateway-1 Started 24.1s
✔ Container mexi-banco-v06-cuenta-1 Started 22.8s
✔ Container mexi-banco-v06-notificacion-1 Started 22.9s
✔ Container mexi-banco-v06-spei-1 Started 22.7s
✔ Container mexi-banco-v06-movimiento-1 Started 22.7s
✔ Container mexi-banco-v06-transferencia-1 Started 22.8s

mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ AC

mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
mexi-banco-v06-cuenta-1	mexi-banco-v06-cuenta	"java -jar app.jar"	cuenta	15 minutes ago	Up 14 minutes (healthy)
0.0.0.0:8081->8080/tcp	postgis:17-alpine	"docker-entrypoint.s..."	db	15 minutes ago	Up 15 minutes (healthy)
5432/tcp					
mexi-banco-v06-discovery-1	mexi-banco-v06-discovery	"java -jar app.jar"	discovery	15 minutes ago	Up 15 minutes (healthy)
0.0.0.0:8761->8761/tcp					
mexi-banco-v06-gateway-1	mexi-banco-v06-gateway	"java -jar app.jar"	gateway	15 minutes ago	Up 14 minutes (healthy)
0.0.0.0:8080->8080/tcp					
mexi-banco-v06-movimiento-1	mexi-banco-v06-movimiento	"java -jar app.jar"	movimiento	15 minutes ago	Up 14 minutes (healthy)
0.0.0.0:8082->8080/tcp					
mexi-banco-v06-notificacion-1	mexi-banco-v06-notificacion	"java -jar app.jar"	notificacion	15 minutes ago	Up 14 minutes (healthy)
0.0.0.0:8084->8080/tcp					
mexi-banco-v06-spei-1	mexi-banco-v06-spei	"java -jar app.jar"	spei	15 minutes ago	Up 14 minutes (healthy)
0.0.0.0:8085->8080/tcp					
mexi-banco-v06-transferencia-1	mexi-banco-v06-transferencia	"java -jar app.jar"	transferencia	15 minutes ago	Up 14 minutes (healthy)
0.0.0.0:8083->8080/tcp					

```

mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
$

```

Figura 2.2. Volviendo a construir el proyecto tras editar los archivos.

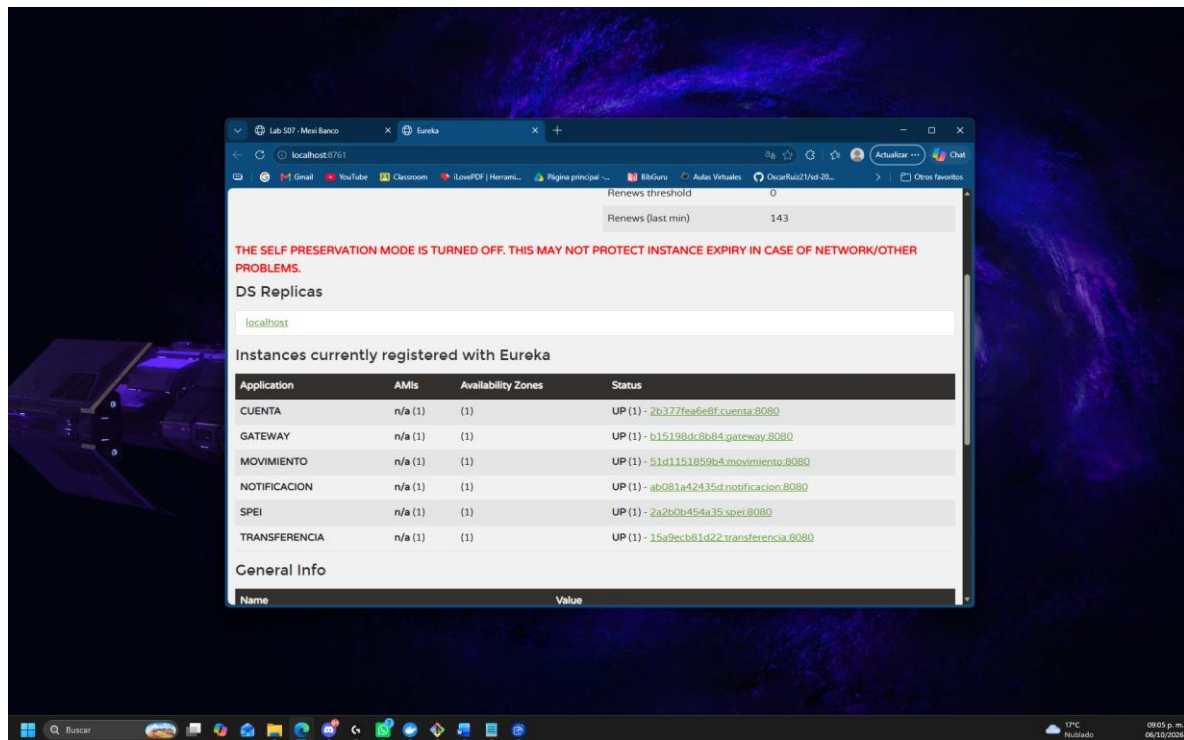


Figura 2.3. Tablero de Eureka.

```
$ AC
mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose ps
NAME                                IMAGE                                COMMAND
PORTS
mexi-banco-v06-cuenta-1            mexi-banco-v06-cuenta              "java -jar d
0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
mexi-banco-v06-db-1                postgres:17-alpine                 "docker-entr
5432/tcp
mexi-banco-v06-discovery-1         mexi-banco-v06-discovery           "java -jar d
0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
mexi-banco-v06-gateway-1          mexi-banco-v06-gateway             "java -jar d
0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-v06-movimiento-1        mexi-banco-v06-movimiento          "java -jar d
0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp
mexi-banco-v06-notificacion-1      mexi-banco-v06-notificacion        "java -jar d
0.0.0.0:8084->8080/tcp, [::]:8084->8080/tcp
mexi-banco-v06-spei-1             mexi-banco-v06-spei               "java -jar d
0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp
mexi-banco-v06-transferencia-1     mexi-banco-v06-transferencia      "java -jar d
0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp

mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose stop notificacion
[+] stop 1/1
✔ Container mexi-banco-v06-notificacion-1 Stopped
```

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (1)	(1)	UP (1) - 2b377fea6e8fcuenta.8080
GATEWAY	n/a (1)	(1)	UP (1) - b15198dc8b84.gateway.8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 51d1151859b4.movimiento.8080
NOTIFICACION	n/a (1)	(1)	DOWN (1) - ab081a42435d.notificacion.8080
SPEI	n/a (1)	(1)	UP (1) - 2a2b0b454a35.spei.8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 15a9ecb81d22.transferencia.8080

General Info

Name	Value
------	-------

Figura 2.4. Deteniendo el servicio notificación.

3.- El gateway: una sola puerta 07/10/2026 – 8:57am

En esta parte agregué el API Gateway como punto de entrada para las peticiones hacia los servicios. Antes de levantarlo hice una prueba directa contra el servicio de CUENTA, donde obtuve una respuesta 400. Después levanté el gateway y comprobé que también aparecía registrado en Eureka.

Al hacer las pruebas mediante el gateway pude observar dos respuestas 404 diferentes. Una fue generada por el servicio de CUENTA al no encontrar el recurso solicitado y la otra fue generada por el propio gateway al no encontrar una ruta que coincidiera con la petición. Esto permitió comprobar que el gateway no solamente recibe las peticiones, sino que también decide a qué servicio y ruta enviarlas.

Después creé una cuenta mediante el gateway y obtuve una respuesta 201. Finalmente consulté la cuenta creada y obtuve una respuesta 200 con sus datos.

```
mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
Content-Type: application/problem+json
Transfer-Encoding: chunked
Date: Wed, 07 Oct 2026 14:48:40 GMT
Connection: close

{"type":"about:blank","title":"Bad Request","status":400,"detail":"Failed to read request","instance":"/movimientos"}
mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
```

Figura 3.1. Prueba antes de encender Gateway.

```
#31 [gateway] resolving provenance for metadata file
#31 DONE 0.1s
[+] up 4/4
✔ Image mexi-banco-v06-discovery Built 39.6s
✔ Image mexi-banco-v06-gateway Built 39.6s
✔ Container mexi-banco-v06-discovery-1 Healthy 4.7s
✔ Container mexi-banco-v06-gateway-1 Started 5.3s

mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ |
```

Figura 3.2. Encendiendo el servicio Gateway.

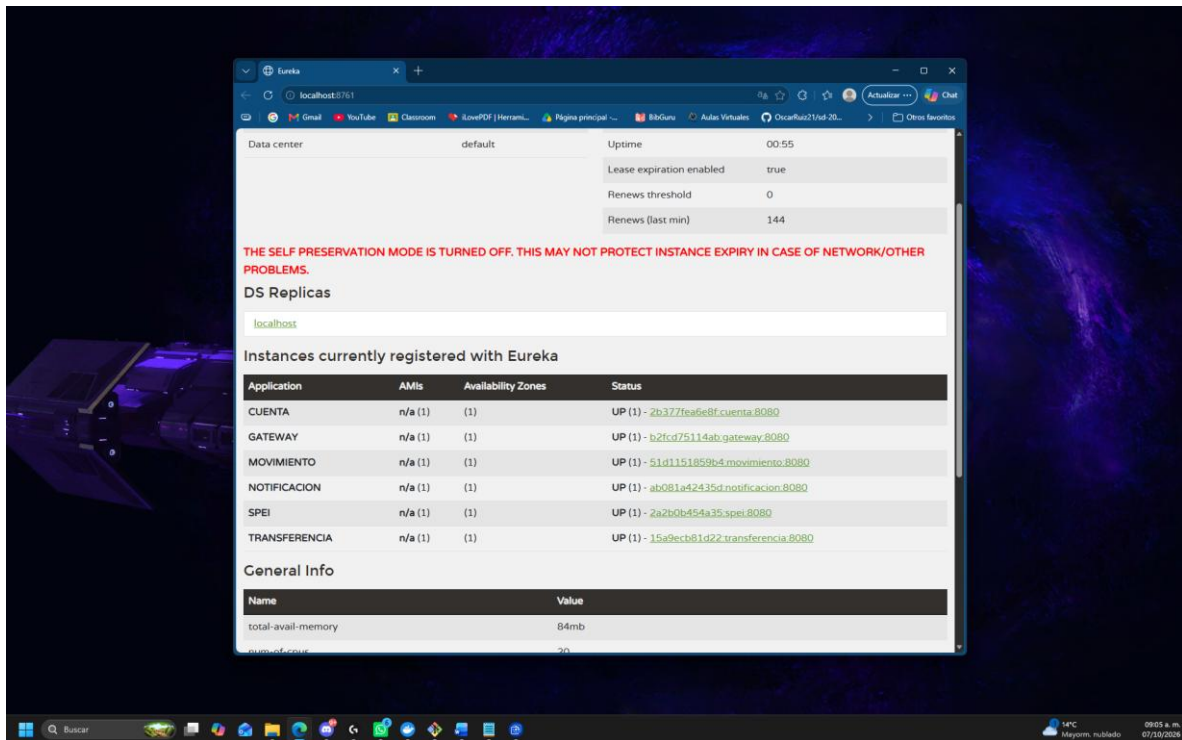


Figura 3.3. Gateway presente en eureka.

```
mauri@ASUSFI5-M MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 2b377fea6e8f
Content-Type: application/problem+json
Date: Wed, 07 Oct 2026 15:07:31 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
mauri@ASUSFI5-M MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 130

{"timestamp":"2026-10-07T15:07:37.944+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"f03b11e5-70"}
mauri@ASUSFI5-M MINGW64 ~/mexi-banco (trabajo-s07)
```

Figura 3.4. Pidiendo una cuenta y registrando movimientos por Gateway.

```

mauri@ASUSFL15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ LAB_CLABE="S07$(date +%s)"

mauri@ASUSFL15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ echo "$LAB_CLABE"
S071791385942

mauri@ASUSFL15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8080/api/cuentas \
  -H 'Content-Type: application/json' \
  -d '{"clabe":"$LAB_CLABE","titular":"S07","saldoInicial":0}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 2b377fea6e8f
Content-Type: application/json
Date: Wed, 07 Oct 2026 15:13:07 GMT
{"id":1,"clabe":"S071791385942","titular":"S07","saldo":0}

mauri@ASUSFL15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ URL="http://localhost:8080/api/cuentas/$LAB_CLABE"

mauri@ASUSFL15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i "$URL"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 2b377fea6e8f
Content-Type: application/json
Date: Wed, 07 Oct 2026 15:14:05 GMT
{"id":1,"clabe":"S071791385942","titular":"S07","saldo":0.00}

mauri@ASUSFL15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ echo "$URL"
http://localhost:8080/api/cuentas/S071791385942

```

Figura 3.5. Creando una cuenta por Gateway.

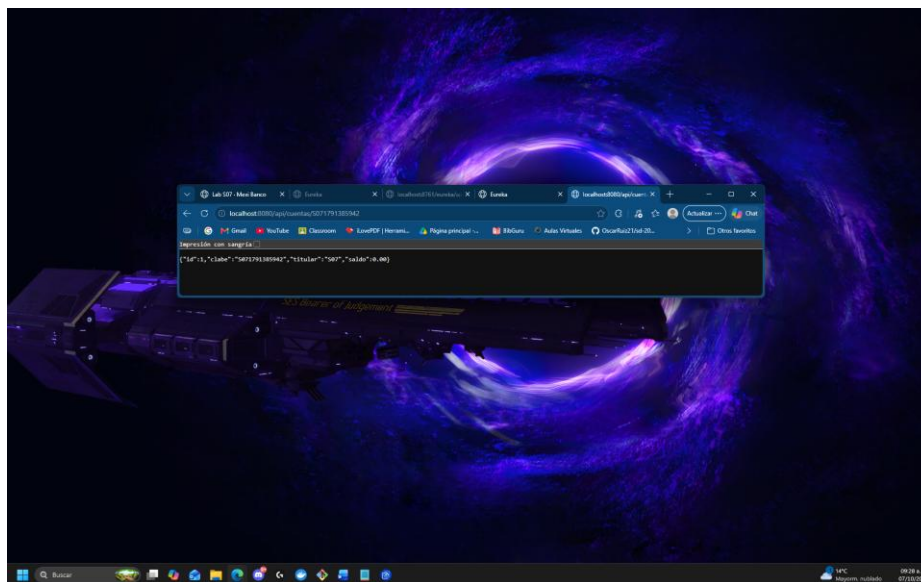


Figura 3.6. Observando cuenta creada en localhost.

4.- Tres copias de cuenta 07/10/2026 – 9:31am

En esta parte configuraré tres réplicas del servicio CUENTA. A diferencia de los otros servicios, las copias de CUENTA ya no tienen un puerto publicado directamente en el equipo, por lo que las peticiones se realizan mediante el gateway.

Después de levantar las tres copias, Eureka mostró CUENTA UP (3). Realicé nueve consultas consecutivas a la misma cuenta y observé la cabecera X-Instancia. Las

respuestas fueron atendidas por las tres copias, siguiendo un comportamiento de tipo Round Robin.

También revisé los logs de CUENTA desde otra terminal para comprobar que las diferentes instancias estaban recibiendo las peticiones.

```
mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ docker ps --filter name=cuenta --format '{{.ID}} {{.Names}} {{.Ports}}'
140ccd1434d0 mexi-banco-v06-cuenta-1 8080/tcp
b67e90b52704 mexi-banco-v06-cuenta-3 8080/tcp
1e73b99ef49f mexi-banco-v06-cuenta-2 8080/tcp
```

Figura 4.1. Consultando las tres copias de cuenta después de crearlas.

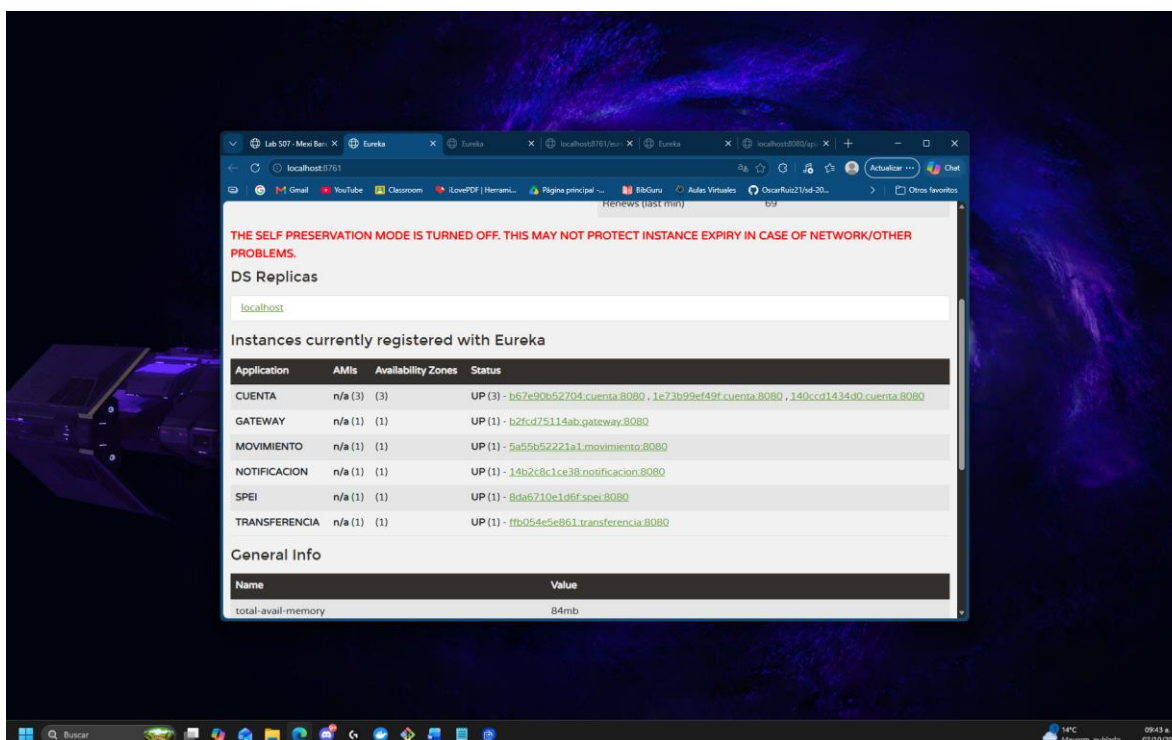


Figura 4.2. Consultando las tres copias de cuenta en Eureka.

```
mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
$ for i in {1..9}; do
  curl -sS -D - -o /dev/null "$URL" | grep -i 'X-Instancia'
done
X-Instancia: b67e90b52704
X-Instancia: 140ccd1434d0
X-Instancia: 1e73b99ef49f
X-Instancia: b67e90b52704
X-Instancia: 140ccd1434d0
X-Instancia: 1e73b99ef49f
X-Instancia: b67e90b52704
X-Instancia: 140ccd1434d0
X-Instancia: 1e73b99ef49f
mauri@ASUSF15-M MINGW64 ~/mexi-banco (trabajo-s07)
```

Figura 4.3. Ciclo de consulta tipo Round Robin.

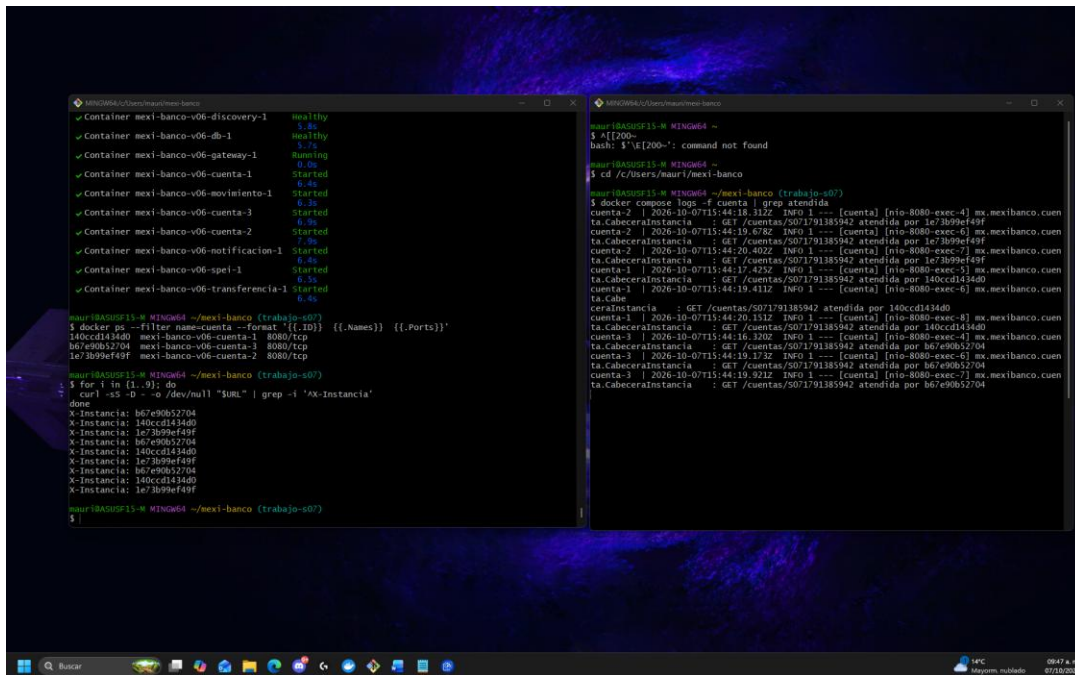


Figura 4.4. Consultando las copias de cuenta en otra terminal.

5.- ¿Y si una copia se muere? 07/10/2026 – 9:52am

Predicción: No van a fallar todas las consultas. Algunas podrían fallar de manera intermitente, aproximadamente una de cada tres, porque inicialmente el gateway todavía considera disponibles a las tres réplicas. Los errores deberían desaparecer cuando el gateway deje de utilizar la instancia que murió y redistribuya las peticiones entre las instancias que siguen disponibles.

¿Qué ocurrió?: Al ejecutar `docker kill` sobre cuenta-1 a las 10:09:46, las peticiones continuaron respondiendo normalmente en la mayoría de los casos, aunque comenzaron a presentarse errores 500 a partir de las 10:09:58. El último error se observó aproximadamente a las 10:10:10. Posteriormente, alrededor de las 10:11, Eureka mostró que el servicio CUENTA había pasado de UP (3) a UP (2).

Esto demuestra que existe un periodo durante el cual el gateway todavía puede intentar enviar solicitudes a una instancia que ya no está disponible, debido a los tiempos de heartbeat, expiración, actualización del registro y caché del balanceador. Una vez actualizada la información, las peticiones pudieron continuar utilizando las otras dos instancias.

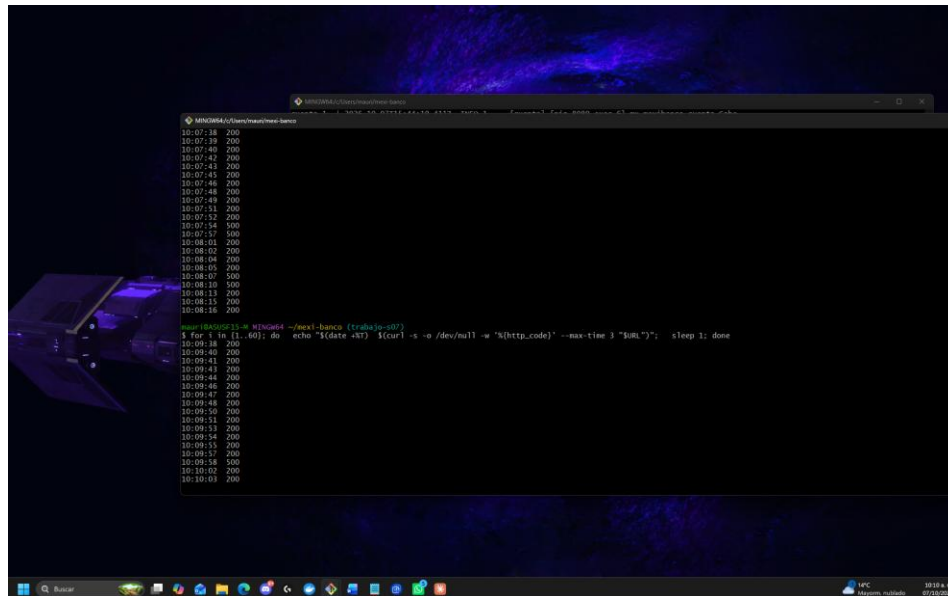


Figura 5.1. Primer error registrado.

6.- Cierre 07/10/2026 – 10:15am

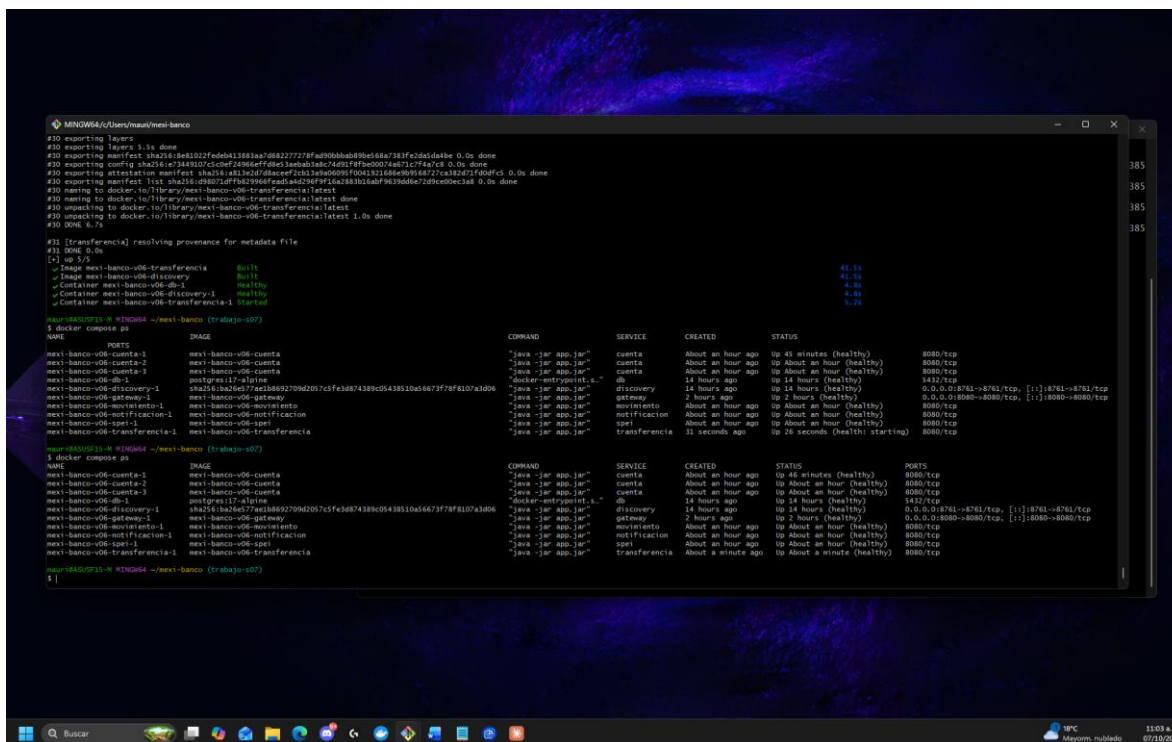
El directorio resuelve el problema de saber qué servicios están disponibles y dónde encontrarlos sin tener que poner sus direcciones directamente. El gateway resuelve el problema de tener que acceder a cada servicio por separado, ya que funciona como una sola puerta de entrada y decide qué peticiones puede recibir cada servicio. El balanceador resuelve el problema de repartir las peticiones cuando existen varias copias de un mismo servicio, haciendo que las solicitudes se distribuyan entre ellas.

Cuando murió una copia, vi que algunas peticiones empezaron a fallar mientras el sistema todavía detectaba la instancia como disponible, porque el gateway todavía tenía registrada la copia que había muerto y la información todavía no se había actualizado. Después de un tiempo, el directorio y el balanceador actualizaron la información y las peticiones dejaron de enviarse a esa instancia.

Tarea: que transferencia también reparta 07/10/2026 – 10:52am

Para esta tarea modifiqué el cliente HTTP de transferencia para que utilizara un RestClient.Builder con @LoadBalanced. Con esto, transferencia puede utilizar el nombre cuenta y consultar al directorio para saber a qué instancia enviar cada petición, en lugar de depender directamente del DNS de Docker.

También cambié las URLs utilizadas por transferencia para quitar el puerto y utilizar únicamente el nombre de cada servicio. Después reconstruí únicamente el servicio de transferencia y ejecuté la demostración.



```
MINGW64 ~/Documents/next-banco
$ docker compose ps
NAME                PORTS                IMAGE                COMMAND                SERVICE                CREATED                STATUS                PORTS
next-banco-v06-cuenta-1    next-banco-v06-cuenta-1    next-banco-v06-cuenta-1    "java -jar app.jar"    cuenta                About an hour ago    up 41 minutes (healthy)    8080/tcp
next-banco-v06-cuenta-2    next-banco-v06-cuenta-2    next-banco-v06-cuenta-2    "java -jar app.jar"    cuenta                About an hour ago    up About an hour (healthy)    8080/tcp
next-banco-v06-cuenta-3    next-banco-v06-cuenta-3    next-banco-v06-cuenta-3    "java -jar app.jar"    cuenta                About an hour ago    up About an hour (healthy)    8080/tcp
next-banco-v06-db-1        postgres117-alpine        sha256:b2d6e77ac3b89270d2057c3fc3d874389c05438510a56673f78f8107a3d06    "docker-entrypoint.s..."    db                14 hours ago        up 14 hours (healthy)    5432/tcp
next-banco-v06-discovery-1    next-banco-v06-discovery-1    sha256:b2d6e77ac3b89270d2057c3fc3d874389c05438510a56673f78f8107a3d06    "java -jar app.jar"    discovery            2 hours ago        up 2 hours (healthy)    0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
next-banco-v06-movimiento-1    next-banco-v06-movimiento-1    next-banco-v06-movimiento-1    "java -jar app.jar"    movimiento            About an hour ago    up About an hour (healthy)    8080/tcp
next-banco-v06-notificacion-1    next-banco-v06-notificacion-1    next-banco-v06-notificacion-1    "java -jar app.jar"    notificacion            About an hour ago    up About an hour (healthy)    8080/tcp
next-banco-v06-spei-1        next-banco-v06-spei-1        next-banco-v06-spei-1        "java -jar app.jar"    spei                About an hour ago    up About an hour (healthy)    8080/tcp
next-banco-v06-transferencia-1    next-banco-v06-transferencia-1    next-banco-v06-transferencia-1    "java -jar app.jar"    transferencia            31 seconds ago        up 26 seconds (health starting)    8080/tcp

$ docker compose up
NAME                IMAGE                COMMAND                SERVICE                CREATED                STATUS                PORTS
next-banco-v06-cuenta-1    next-banco-v06-cuenta-1    next-banco-v06-cuenta-1    "java -jar app.jar"    cuenta                About an hour ago    up 46 minutes (healthy)    8080/tcp
next-banco-v06-cuenta-2    next-banco-v06-cuenta-2    next-banco-v06-cuenta-2    "java -jar app.jar"    cuenta                About an hour ago    up About an hour (healthy)    8080/tcp
next-banco-v06-cuenta-3    next-banco-v06-cuenta-3    next-banco-v06-cuenta-3    "java -jar app.jar"    cuenta                About an hour ago    up About an hour (healthy)    8080/tcp
next-banco-v06-db-1        postgres117-alpine        sha256:b2d6e77ac3b89270d2057c3fc3d874389c05438510a56673f78f8107a3d06    "docker-entrypoint.s..."    db                14 hours ago        up 14 hours (healthy)    5432/tcp
next-banco-v06-discovery-1    next-banco-v06-discovery-1    sha256:b2d6e77ac3b89270d2057c3fc3d874389c05438510a56673f78f8107a3d06    "java -jar app.jar"    discovery            14 hours ago        up 14 hours (healthy)    0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
next-banco-v06-gateway-1    next-banco-v06-gateway-1    sha256:b2d6e77ac3b89270d2057c3fc3d874389c05438510a56673f78f8107a3d06    "java -jar app.jar"    gateway                2 hours ago        up 2 hours (healthy)    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
next-banco-v06-movimiento-1    next-banco-v06-movimiento-1    next-banco-v06-movimiento-1    "java -jar app.jar"    movimiento            About an hour ago    up About an hour (healthy)    8080/tcp
next-banco-v06-notificacion-1    next-banco-v06-notificacion-1    next-banco-v06-notificacion-1    "java -jar app.jar"    notificacion            About an hour ago    up About an hour (healthy)    8080/tcp
next-banco-v06-spei-1        next-banco-v06-spei-1        next-banco-v06-spei-1        "java -jar app.jar"    spei                About an hour ago    up About an hour (healthy)    8080/tcp
next-banco-v06-transferencia-1    next-banco-v06-transferencia-1    next-banco-v06-transferencia-1    "java -jar app.jar"    transferencia            About a minute ago    up About a minute (healthy)    8080/tcp
```

Figura 6.1. Reconstruyendo transferencia.

```
MINGW64 C:\Users\maur\mi-banco
cuenta-2 POST /cuentas/002180009139404001/cargos atendida por 1e73b99ef49f
cuenta-1 POST /cuentas/002180009139404002/abonos atendida por 140ccd14340
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

--- 4. Consultar una transferencia ---
$ curl http://localhost:8080/api/transferencias/1
{"id":1,"claveorigen":"002180009139404001","clavedestino":"002180009139404002","monto":200.00,"fecha":"2026-10-07T17:27:30.338483Z"}
[HTTP 200]

--- 5. Saldos: el dinero no se crea ni se pierde ---
$ curl http://localhost:8080/api/cuentas/002180009139404001
{"id":2,"clave":"002180009139404001","titular":"Ana","saldo":800.00}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009139404002
{"id":5,"clave":"002180009139404002","titular":"Beto","saldo":700.00}
[HTTP 200]
-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700

--- 6. Estado de cuenta de Ana y avisos de moto (GET /api/movimientos, GET /api/notificaciones) ---
$ curl http://localhost:8080/api/movimientos?clave=002180009139404001
[{"id":1,"clavecuenta":"002180009139404001","tipo":"TRANSFERENCIA_ENVIADA","monto":200.00,"saldoresultante":800.00,"fecha":"2026-10-07T17:27:30.218234Z","referencia":"a 002180009139404002"}, {"id":3,"clavecuenta":"002180009139404001","tipo":"DEPOSITO","monto":1000.00,"saldoresultante":1000.00,"fecha":"2026-10-07T17:27:16.476671Z","referencia":"saldo inicial"}]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clave=002180009139404002
{"id":2,"clavecuenta":"002180009139404002","mensaje":"TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.00","fecha":"2026-10-07T17:27:35.123729Z"}
[HTTP 200]

--- 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ---
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -d '{"idempotency-key": "spei-demo-91394040" -d '{"claveorigen":"002180009139404001","clavedestino":"BANCO-X","clavedestino":"00218000000000000009","monto":50}
[{"id":3,"idempotencykey":"spei-demo-91394040","claveorigen":"002180009139404001","clavedestino":"BANCO-X","clavedestino":"00218000000000000009","monto":50,"estado":"ENVIADO"}]
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -d '{"idempotency-key": "spei-demo-91394040" -d '{"claveorigen":"002180009139404001","clavedestino":"BANCO-X","clavedestino":"00218000000000000009","monto":50}
[{"id":3,"idempotencykey":"spei-demo-91394040","claveorigen":"002180009139404001","clavedestino":"BANCO-X","clavedestino":"00218000000000000009","monto":50.00,"estado":"ENVIADO"}]
[HTTP 201]
-> mismo id (3 -> 3): no se cobro dos veces, saldo de Ana: 750.00 (esperado 750)

--- 8. maur robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ---
consulta 1 X-Instancia: 1e73b99ef49f (mexi-banco-v06-cuenta-2)
consulta 2 X-Instancia: 1e73b99ef49f (mexi-banco-v06-cuenta-2)
consulta 3 X-Instancia: 140ccd14340 (mexi-banco-v06-cuenta-1)
consulta 4 X-Instancia: b67e90b52704 (mexi-banco-v06-cuenta-3)
consulta 5 X-Instancia: 1e73b99ef49f (mexi-banco-v06-cuenta-2)
consulta 6 X-Instancia: 140ccd14340 (mexi-banco-v06-cuenta-1)
consulta 7 X-Instancia: b67e90b52704 (mexi-banco-v06-cuenta-3)
consulta 8 X-Instancia: 1e73b99ef49f (mexi-banco-v06-cuenta-2)
consulta 9 X-Instancia: b67e90b52704 (mexi-banco-v06-cuenta-3)
-> 9 replicas distintas en 9 consultas; 1 veces se repitió la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan solo conoce el 8080

--- 9. Los endpoints internos no tienen puerta ---
$ curl -X POST http://localhost:8080/api/cuentas/002180009139404001/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{"timestamp":"2026-10-07T17:27:50.703400+00","path":"/api/cuentas/002180009139404001/abonos","status":404,"error":"not Found","requestid":"c8be6bb1-1953"}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

--- 10. Errores de negocio, por la puerta, con el mismo código que dio cuenta ---
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveorigen":"002180009139404001","clavedestino":"002180009139404099","monto":10}'
{"type":"about:blank","title":"not Found","status":404,"detail":"No existe la CLAVE 002180009139404099","instance":"/transferencias","servicio":"cuenta"}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveorigen":"002180009139404001","clavedestino":"002180009139404002","monto":999999}'
{"type":"about:blank","title":"unprocessable Entity","status":422,"detail":"saldo insuficiente en la CLAVE 002180009139404001","instance":"/transferencias","servicio":"cuenta"}
[HTTP 422]

--- 11. Los saldos no se movieron con los errores ---
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido
$
$
```

Figura 6.2. Ejecutando prueba.

La demostración se ejecutó correctamente. Primero se comprobó que Eureka conocía las tres instancias de CUENTA y que los demás servicios también estaban registrados. Después se crearon las cuentas de Ana con \$1000 y Beto con \$500, ambas mediante el gateway.

La transferencia de \$200 se realizó correctamente y devolvió un 201. Lo más importante de esta prueba fue observar que las diferentes llamadas realizadas por transferencia fueron atendidas por distintas copias de CUENTA: la consulta de Ana fue atendida por cuenta-1, la consulta de Beto por cuenta-3, el cargo por cuenta-2 y el abono por cuenta-1. Esto comprueba que transferencia ahora utiliza el directorio y su balanceador para repartir las peticiones.

Después de la transferencia, los saldos quedaron en \$800 para Ana y \$700 para Beto. También se comprobó la consulta de la transferencia, el movimiento registrado en la cuenta de Ana y la notificación recibida por Beto.

Posteriormente se realizó un SPEI de \$50. Al repetirlo utilizando el mismo Idempotency-Key, se obtuvo el mismo registro y no se realizó un segundo cobro. El saldo de Ana quedó en \$750, como se esperaba.

Finalmente, las nueve consultas consecutivas mostraron las tres réplicas de CUENTA, comprobando nuevamente el balanceo. Los errores de negocio también se

comportaron como se esperaba: una CLABE inexistente produjo un 404 y un saldo insuficiente produjo un 422, sin modificar los saldos. Al terminar, Ana tenía \$750 y Beto \$700.